증상별 RAG 검색과 근거를 확인하는 보고서

과제 1 제공된 운영 인시던트 대응 에이전트 개선

기존 기반 활용: FastAPI, Vue, LangGraph, MCP, LangSmith

교육용 모니터링 데이터로 조사하며 실제 조치는 실행하지 않습니다

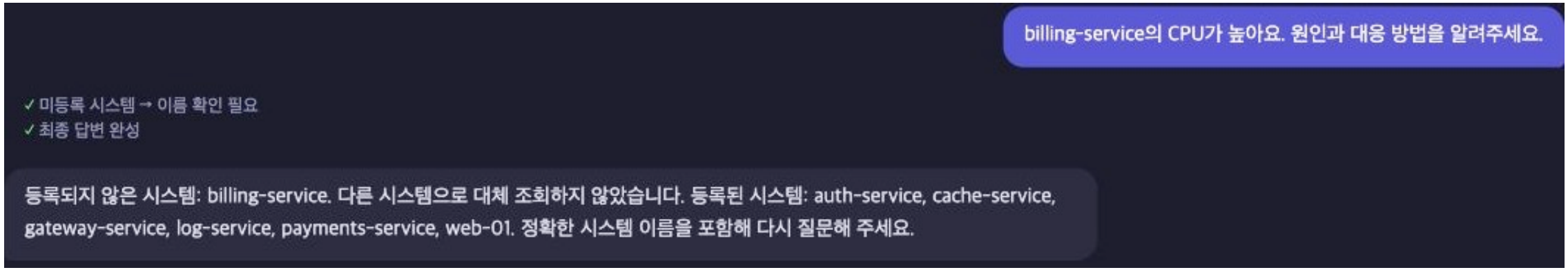
직접 추가·수정한 기능

조사 대상·복수 시스템 처리 보완

미등록·미지정 대상 확인, 작업별 실패 기록과 성공 자료 보존

런북 검색·답변 검증·보고서 화면 개선

증상별 RAG, 제외 조건, 조치 조건 검사와 재작성, 요약·상세 분리



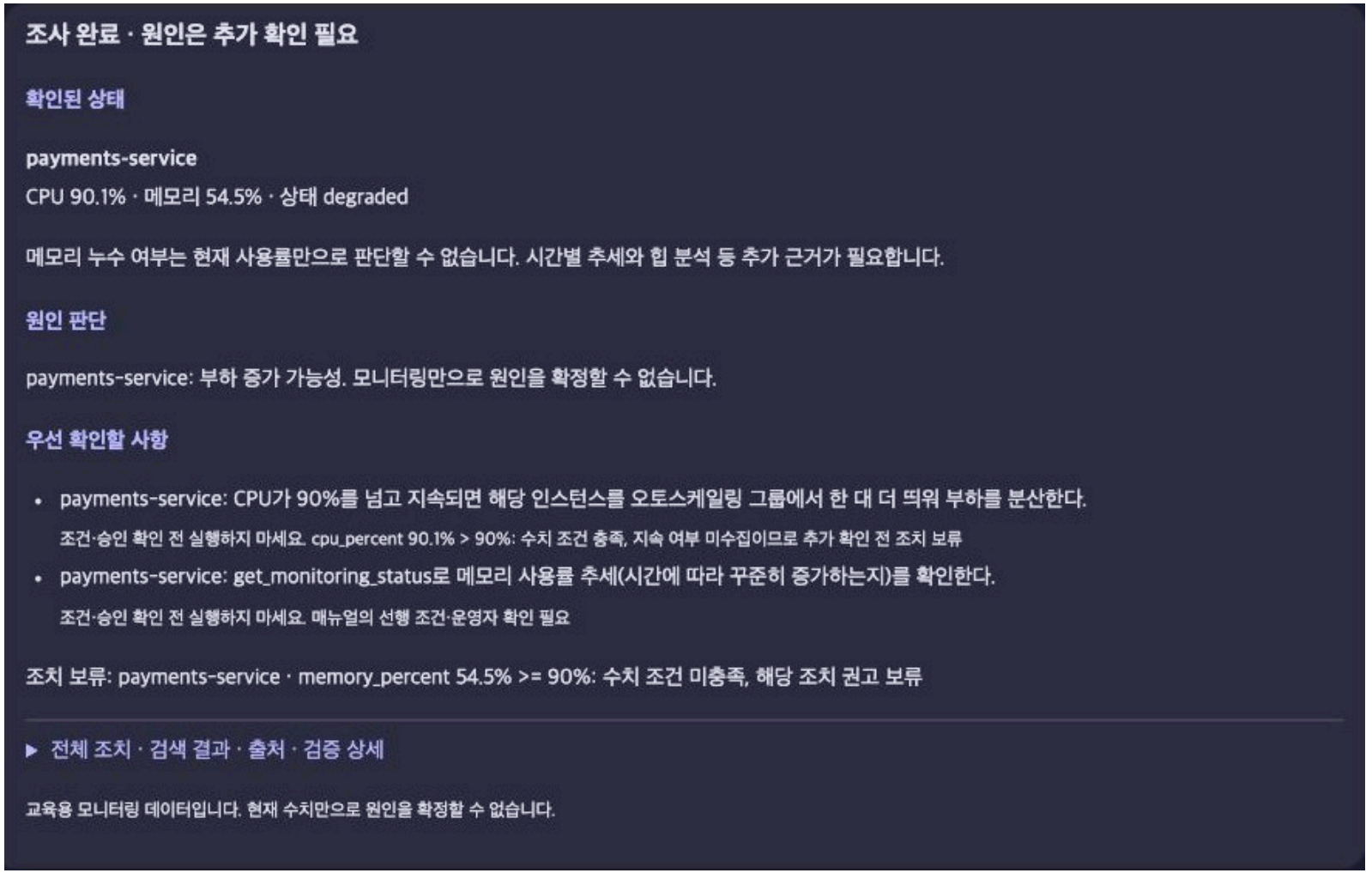
CPU·메모리 복합 질문 결과

선택한 런북

cpu-spike
memory-leak

현재 메모리 54.5%
90% 조치 조건 미충족
재시작 권고 보류

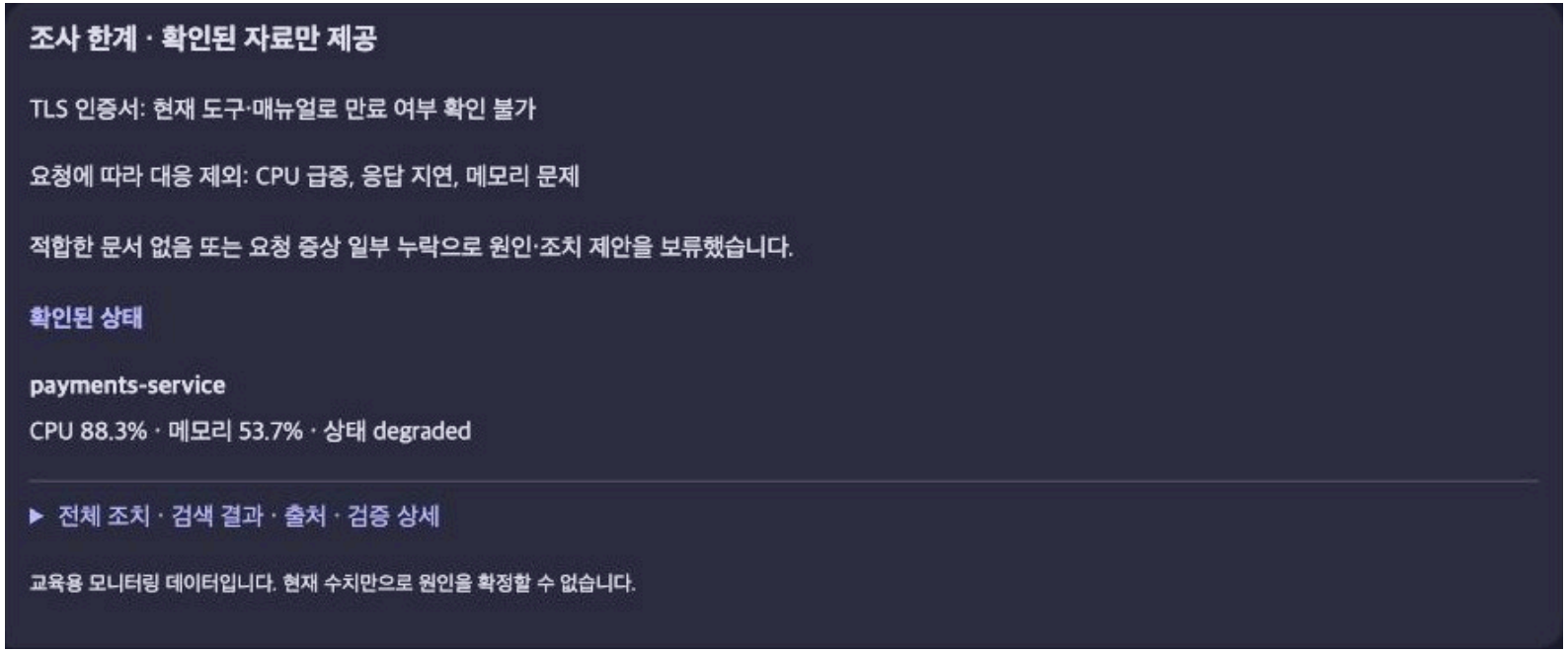
원인 확정과 실행 승인은
별도 판단이 필요합니다



TLS 질문의 조사 한계 안내

현재 도구로
인증서 만료 확인 불가

관련 매뉴얼 없음
제외한 대응 추천 안 함
원인·조치 생성 생략



확인 가능한 사실과
조사 한계를 함께 표시

Agent Graph와 공통 데이터

graph.py / build_graph()

```
g.add_edge("plan", "gate")
g.add_edge("gate", "research")
g.add_edge("research", "replan_check")
g.add_edge("analysis", "write")
g.add_edge("write", "validate")
```

state 이번 질문의 기록장
질문, 대상, 작업, 수집 자료, 검증 결과

deps 함께 사용하는 도구
모델, MCP 연결, 검색기, 저장소

MCP 조회와 부분 실패 처리

agent/nodes/research.py

```
await deps.mcp.call_tool(
    "get_monitoring_status",
    {"system_name": task["target"]})

results = await asyncio.gather(
    *(run_one(i) for i in research_indices),
    return_exceptions=True)
```

Tool: 상태 조회 Resource: 런북 원문 Prompt: 조사 절차

일부 조회 실패 시 성공 자료 보존, 실패 대상과 진단 한계 표시

증상별 RAG 검색과 범위 제한

agent/runbook_search.py / search_request()

```
for kind in scope["included"]:
    candidates = await self.search(TOPICS[kind][1])
    matches = [h for h in candidates
               if h["incident_type"] == kind]
```

범위 해석: 증상·제외 조건의 표현 규칙
문서 선택: 임베딩 유사도와 최소 점수 조건

증상별 결과를 합치고 원문을 수집해 AI 작성 근거로 전달

문서 누락·미지원 요청·검색 오류를 기본 런북으로 대체하지 않음

검증·재작성과 다음 개선

agent/nodes/writing.py / validate_node()

```
proposal, issues = validate_proposal(
    state["draft"],
    state.get("collected", []),
    state.get("registered_targets", []))
```

출처·대상·조건·요청 범위 검사, 최대 3회 작성

자동 테스트 76개 통과, 대표 실모델 실행 확인
원인 확정과 전체 검색 정확도 검증은 별도

다음 개선: 복잡한 범위 표현, 후속 대화 연결, 동기 호출